

Spring Security

Spring Security

Spring Security is a framework that focuses on providing both authentication and authorization (access control) to java web application and SOAP/REST web services.

Spring framework supports integration with many technologies

- HTTP basic authentication
- LDAP
- OpenID providers
- JAAS API
- ...
- And customized authentication system (by yourself)

Spring Security

Terminologies

Principal

User that performs the action

Authentication

Confirming truth of credentials

Authorization

Define access policy for principal

GrantedAuthority

Application permission granted to a principal

SecurityContext

Hold the authentication and other security information

SecurityContextHolder

Provides access to SecurityContext

▶ AuthenticationManager

▶ Controller in the authentication process

▶ AuthenticationProvider

▶ Interface that maps to a data store which stores your user data.

▶ Authentication Object

▶ Object is created upon authentication, which holds the login credentials.

▶ UserDetails

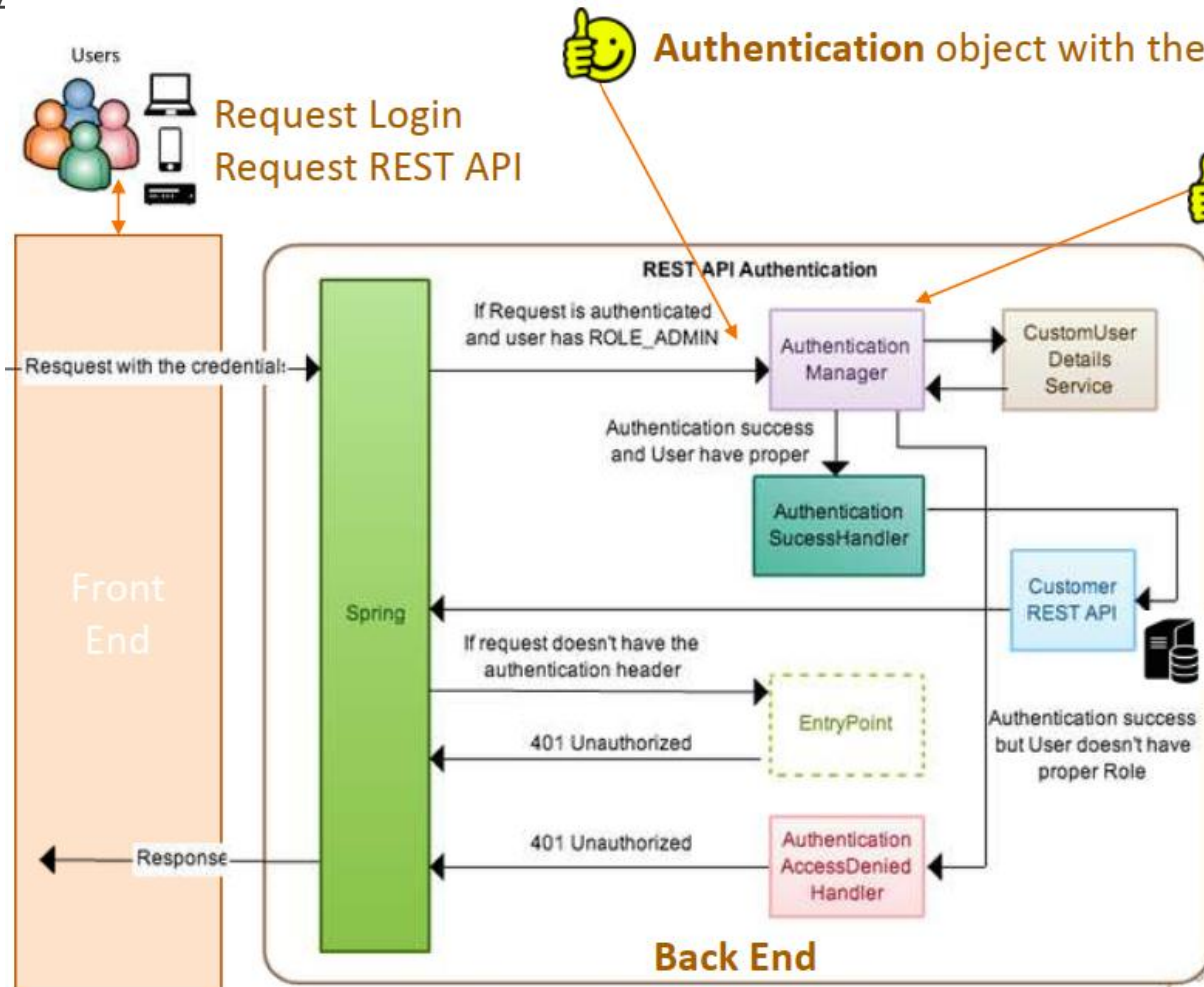
▶ Data object which contains the user credentials, but also the Roles of the user.

▶ UserDetailsService

- Collects the user credentials, authorities(roles) and build an UserDetails object.

Spring Security

Architecture



Authentication object with the login credentials is created



Controller in the process that it just delegates to a collection of **AuthenticationProvider** instances.

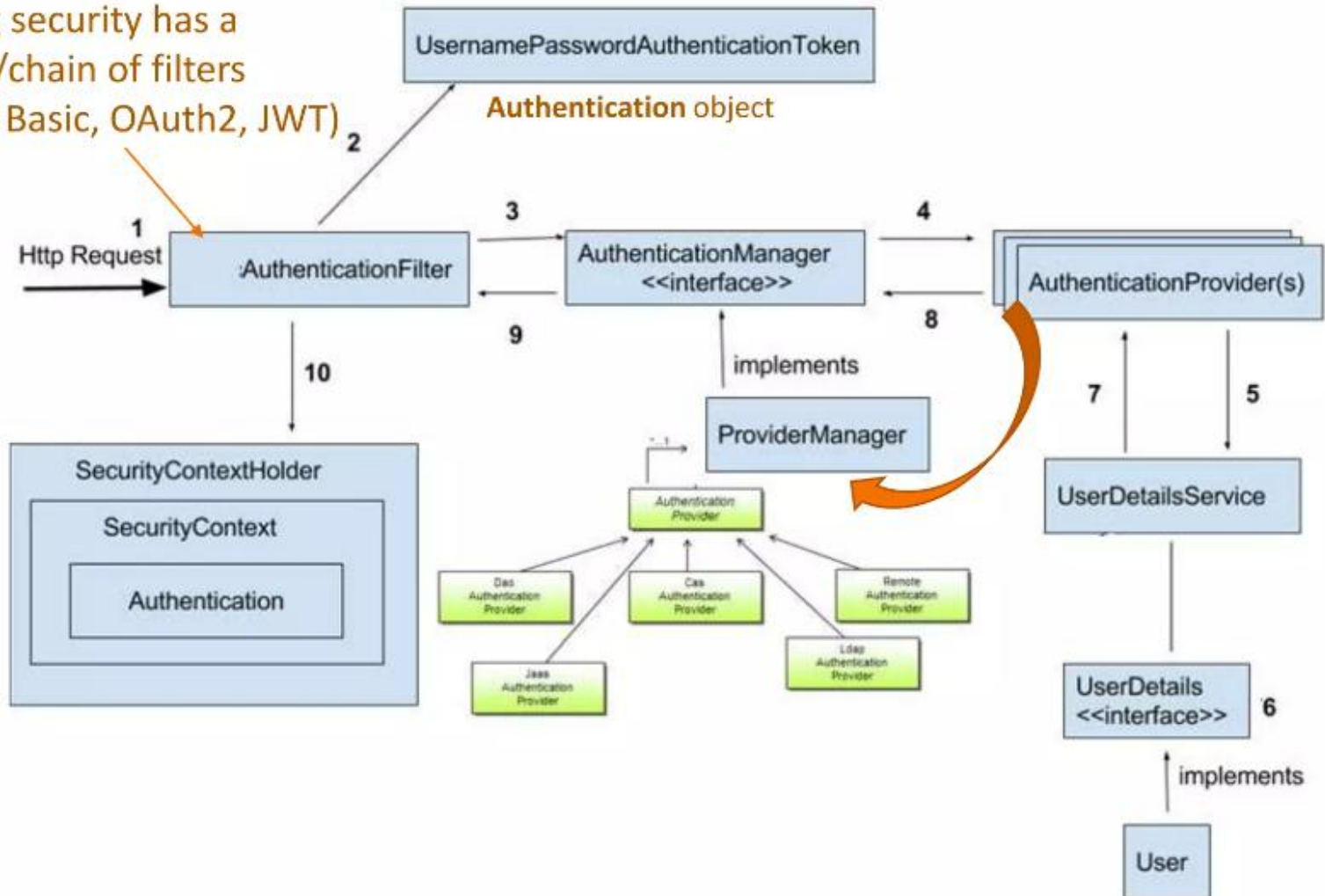
Call an object that implements the **UserDetailsService** interface, which it looks up the user data and returns a **UserDetails** object.

Will check that the password matches the password the user entered.

Spring Security



fl Spring security has a series/chain of filters (HTTP Basic, OAuth2, JWT)



Spring Security

In Spring 6.x, by adding the dependency to project, when you access any resources, you should provide the credential.

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-security</artifactId>  
</dependency>  
<dependency>  
  <groupId>org.springframework.security</groupId>  
  <artifactId>spring-security-test</artifactId>  
  <scope>test</scope>  
</dependency>
```

ion, you

spring.

Sample steps:

- Create any class with the `@Configuration` annotation.
- Add `@EnableWebSecurity`
- Create a method with a parameter with type `AuthenticationManagerBuilder`
- Create a bean with return type `SecurityFilterChain` and a parameter with type `HttpSecurity`

Spring Security

Resources' grant/deny permission

You should specify resources for protecting.

- PermitAll()
- Authenticated()
- denyAll()
- hasAnyRole()
- hasRole()
- ...


```

@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .csrf().disable()
        .authorizeHttpRequests(
            auth -> auth
                .requestMatchers("/","/index", "/error", "/home").permitAll()
                .requestMatchers("/admin", "/admin/**").hasRole(ADMIN_ROLE)
                // .requestMatchers("/api/**").hasRole(USER_ROLE)
                .requestMatchers("/api/**").hasAnyRole(USER_ROLE, ADMIN_ROLE)
                .requestMatchers("/sensitive/**").denyAll()
                // .anyRequest().authenticated()
        )
        .httpBasic(Customizer.withDefaults())
    ;
    return http.build();
}

```

You can provide credential by many ways:

- In memory
- Using JDBC
- Using OAuth2AuthorizeRequest
- Using LDAP
- ...

In this study, in memory and using JDBC credentials are used.

Using `BCryptPasswordEncoder` for encoding the password

Provide role for users

```
@Configuration
@EnableWebSecurity
public class SecurityConfig{

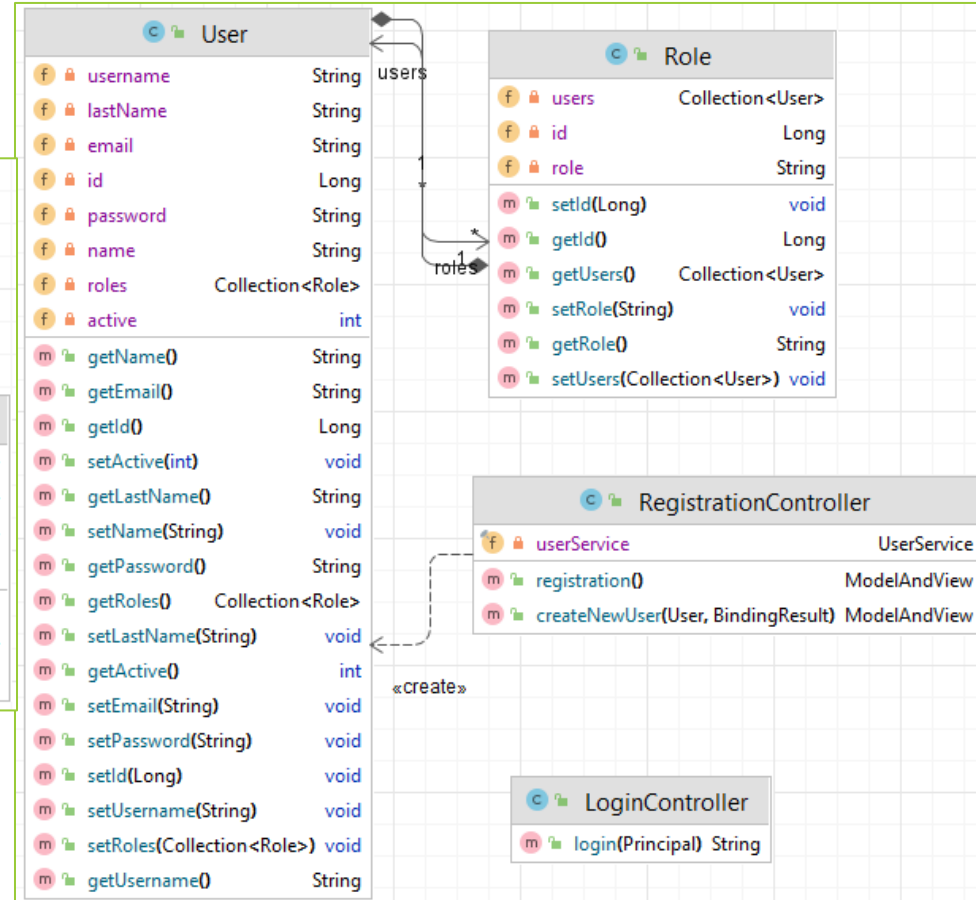
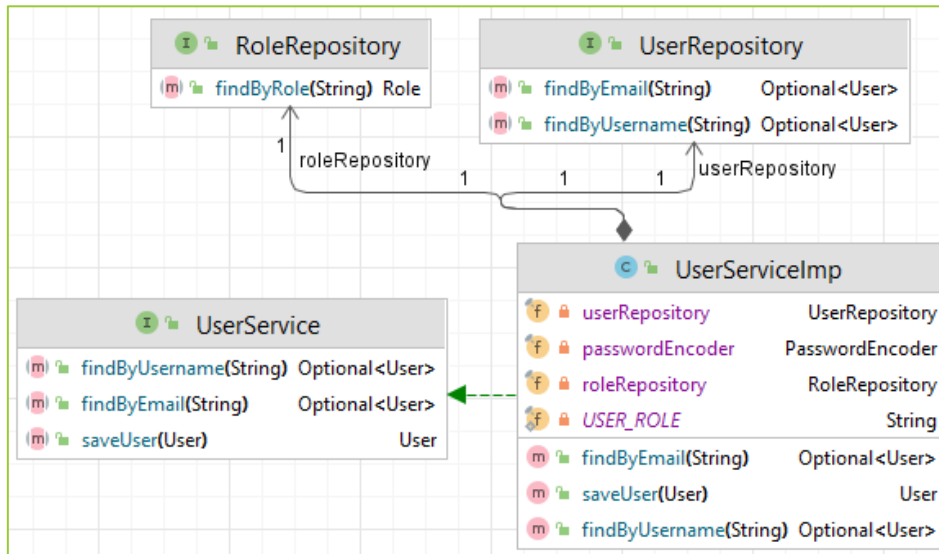
    private static final String USER_ROLE = "USER";
    private static final String ADMIN_ROLE = "ADMIN";

    @Autowired
    public void configureGlobal(AuthenticationManagerBuilder auth) throws Exception {

        UserDetails ud_teo = User.builder()
            .username("teo")
            .password("$2a$10$iVIV6KTf2GdAAo9FpLVpsubzVP8undhidTtCyftx.0nfBn5yn2eZW")
            .roles(ADMIN_ROLE)
            .build();
        UserDetails ud_ty = User.builder()
            .username("ty")
            .password("$2a$10$hFhz/f3e0GI5y4baqAvdX.BLEiMivXXc0U9kKNgyUDCBi1f7hbmFa")
            .roles(USER_ROLE)
            .build();
        auth.inMemoryAuthentication()
            .withUser(ud_teo)
            .withUser(ud_ty);
    }
}
```

Credentials

Using JDBC credentials (1)



Credentials

Using JDBC credentials (2)

```
# Queries for AuthenticationManagerBuilder
spring.queries.users-query=select username, password, active from user where username=?
spring.queries.roles-query=select u.username, r.role from user u inner join user_role ur on(u.user_id=ur.user_id) \
    inner join role r on(ur.role_id=r.role_id) where u.username=?
# Admin username and password
spring.admin.username=admin
spring.admin.password=admin
```

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
    private static final String USER_ROLE = "USER";
    private static final String ADMIN_ROLE = "ADMIN";
    private final DataSource dataSource;
    @Value("${spring.admin.username}")
    private String adminUsername;
    @Value("${spring.admin.password}")
    private String adminPassword;
    @Value("${spring.queries.users-query}")
    private String usersQuery;
    @Value("${spring.queries.roles-query}")
    private String rolesQuery;

    public SecurityConfig(DataSource dataSource) {
        this.dataSource = dataSource;
    }
}
```

```
@Autowired
public void configureGlobal(
    AuthenticationManagerBuilder auth) throws Exception {
    auth.
        jdbcAuthentication()
        .usersByUsernameQuery(usersQuery)
        .authoritiesByUsernameQuery(rolesQuery)
        .dataSource(dataSource)
        .passwordEncoder(new BCryptPasswordEncoder());
}
```

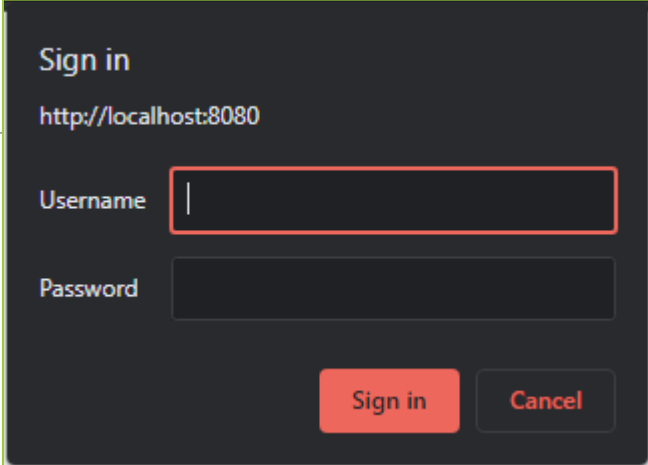
```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Logon Forms

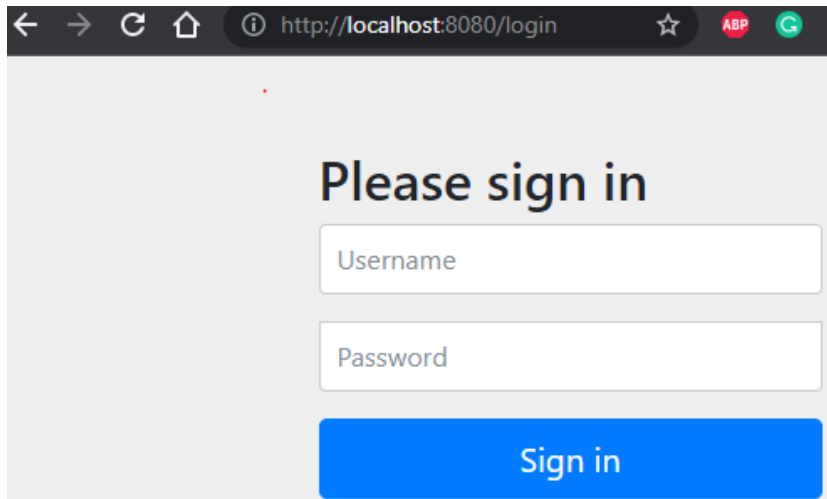
Security form

The logging forms:

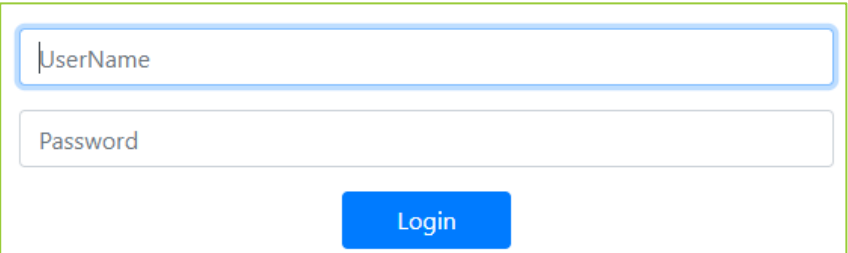
- From web browser (system)
- From spring security
- Custom (specify the logon form by yourself)



A dark-themed sign-in form titled "Sign in" with the URL "http://localhost:8080" below it. It features two input fields: "Username" and "Password". The "Username" field is highlighted with a red border. Below the fields are two buttons: "Sign in" (red) and "Cancel" (gray).



A light-themed sign-in form titled "Please sign in". It features two input fields: "Username" and "Password". Below the fields is a blue button labeled "Sign in". The browser address bar shows "http://localhost:8080/login".



A simple light-themed login form with two input fields: "UserName" and "Password". Below the fields is a blue button labeled "Login".

Logon Forms

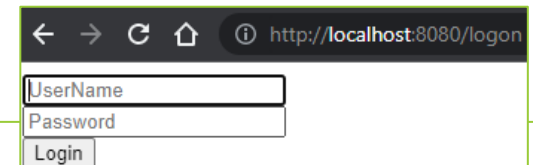
```
implementation 'org.springframework.boot:spring-boot-starter-thymeleaf'  
implementation 'org.thymeleaf.extras:thymeleaf-extras-springsecurity6:3.1.1.RELEASE'
```

Custom logon form

```
<form method="post" th:action="@{/logon}">  
  <div class="form-group">  
    <input autofocus="true" class="form-control" placeholder="User Name" required="" type="text">  
  </div>  
  <div class="form-group">  
    <input class="form-control input-lg" placeholder="Password" required="true" type="password">  
  </div>  
  <div class="row">  
    <div class="col-sm-3" style="text-align: right;">  
      <input class="btn btn-primary" type="submit" value="Logon">  
    </div>  
  </div>  
</form>
```

```
@GetMapping("/logon")  
public String login(Principal principal) {  
    if (principal != null) {  
        return "redirect:/home";  
    }  
    return "/logon";  
}
```

```
@Bean  
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {  
    http  
        .csrf().disable()  
        .authorizeHttpRequests(auth -> auth  
            .requestMatchers("/","/index","/error","/home").permitAll()  
            .requestMatchers("/admin","/admin/**").hasRole(ADMIN_ROLE)  
            //.requestMatchers("/api/**").hasRole(USER_ROLE)  
            .requestMatchers("/api/**").hasAnyRole(USER_ROLE,ADMIN_ROLE)  
            .requestMatchers("/sensitive/**").denyAll()  
            //.anyRequest().authenticated()  
        )  
        .formLogin(form -> form  
            .loginPage("/logon")  
            .defaultSuccessUrl("/home")  
            .permitAll()  
        )  
        .logout(LogoutConfigurer::permitAll)  
        .httpBasic(Customizer.withDefaults())  
    ;  
    return http.build();  
}
```



← → ↻ ⬆ ⓘ http://localhost:8080/logon

UserName
Password
Login

Method level security

Method's Security

In case of specifying roles for accessing methods, you should enable method security.

Using `@EnableMethodSecurity` annotation on any `@Configuration` class

```
@RestController
@RequestMapping("/api")
public class ProductResources {
    @GetMapping("/test")
    public ResponseEntity<Student> getTest() {
        ResponseEntity<Student> s = ResponseEntity.ok(new Student( id: 1001, name: "Nguyen van teo"));
        return s;
    }

    @PreAuthorize("hasRole('ADMIN')")
    @GetMapping("/admin")
    public ResponseEntity<Student> testAdmin() {
        ResponseEntity<Student> s = ResponseEntity.ok(new Student( id: 1011, name: "Administrator"));
        return s;
    }
}
```

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {
```


Other techniques

OAuth2 (Single sign-On)

JSON Web Token (JWT)

OpenID Connect

...

Cross-Site Request Forgery (CSRF)

Introduction

Cross-Site Request Forgery (CSRF) is an attack that forces an end user to execute unwanted actions on a web application in which they're currently authenticated.

With a little help of social engineering (such as sending a link via email or chat), an attacker may trick the users of a web application into executing actions of the attacker's choosing.

If the victim is a normal user, a successful CSRF attack can force the user to perform state changing requests like transferring funds, changing their email address, and so forth.

If the victim is an administrative account, CSRF can compromise the entire web application.

There are multiple forms of CSRF attack. Two simple CSRF attacks that most common happen:

- Using GET method
- Using POST method

CSRF attacks

Using get method

Let's consider the following GET request used by a logged-in user to transfer money to a specific bank account 1234

GET <http://bank.com/transfer?accountNo=1234&amount=100>

If the attacker wants to transfer money from a victim's account to his own account instead — 5678 — he needs to make the victim trigger the request:

GET <http://bank.com/transfer?accountNo=5678&amount=1000>

There are multiple ways to make that happen:

- Link – The attacker can convince the victim to click on this link, for example, to execute the transfer:
- Image – The attacker may use an `<img/>` tag with the target URL as the image source. In other words, the click isn't even necessary. The request will be automatically executed when the page loads:

```
<a href="http://bank.com/transfer?accountNo=5678&amount=1000">  
  Show Kittens Pictures  
</a>
```

```

```

CSRF attacks

Using POST method

Suppose the main request needs to be a POST request:

```
POST http://bank.com/transfer
accountNo=1234&amount=100
```

In this case, the attacker needs to have the victim run a similar request:

```
POST http://bank.com/transfer
accountNo=5678&amount=1000
```

Neither the `<a>` nor the `<img>` tags will work in this case.

The attacker will need a `<form>`:

```
<form action="http://bank.com/transfer" method="POST">
  <input type="hidden" name="accountNo" value="5678"/>
  <input type="hidden" name="amount" value="1000"/>
  <input type="submit" value="Show Kittens Pictures"/>
</form>
```

The form can be submitted automatically using

JavaScript:

```
<body onload="document.forms[0].submit()">
<form>
...
</form>
```

Spring MVC web with CSRF

Prevent CSRF attack

To protect MVC applications, Spring adds a CSRF token to each generated view.

This token must be submitted to the server on every HTTP request that modifies state (PATCH, POST, PUT and DELETE — not GET).

This protects our application against CSRF attacks since an attacker can't get this token from their own page.

Starting from Spring Security 4.x, the CSRF protection is **enabled by default**.

This default configuration adds the CSRF token to the HttpServletRequest attribute named `_csrf`.

If we need to, we can disable this configuration but not recommended

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .csrf().disable();
    return http.build();
}
```

Spring MVC web with CSRF

Client prevent CSRF attack configuration

We need to include the CSRF token in our requests. The `_csrf` attribute contains the following information:

- *token* – the CSRF token value
- *parameterName* – name of the HTML form parameter, which must include the token value
- *headerName* – name of the HTTP header, which must include the token value

In case of using HTML forms, the *headerName* and *token* values should be added HTTP header.

In case of using JSON view forms, the *headerName* and *token* values should be added HTTP header.

```
<input type="hidden" name="${_csrf.parameterName}" value="${_csrf.token}"/>
```

```
//1. include the token value and the header name in meta tags  
<meta name="_csrf" content="${_csrf.token}"/>  
<meta name="_csrf_header" content="${_csrf.headerName}"/>
```

```
//2. Then let's retrieve the meta tag values with JQuery:  
var token = $("meta[name='_csrf']").attr("content");  
var header = $("meta[name='_csrf_header']").attr("content");
```

```
// 3. Finally, let's use these values to set our XHR header:  
$(document).ajaxSend(function (e, xhr, options) {  
    xhr.setRequestHeader(header, token);  
});
```

Control the Session with Spring Security

Control the Session with Spring Security

<https://www.baeldung.com/spring-security-session>
